

Out-of-bounds Write

Escritura fuera de límites

Desarrollo Seguro de Software • Juan Ignacio Mato



**Escribir fuera de los límites del
buffer reservado**



¿Cómo funciona?

El mecanismo

- Un buffer tiene un tamaño FIJO (ej.: 16 bytes).
- En memoria, las variables viven pegadas una al lado de la otra.
- Se copian más datos de los que entran y **nadie controla el límite**.
- La escritura se “desborda” y sigue pisando la memoria vecina.

¿Qué se puede pisar?

- Otra variable: un precio, un flag “es admin”.
- La dirección de retorno → redirigir la ejecución.
- Punteros y metadatos del heap.
- Resultado: desde datos corruptos y crashes hasta **ejecución de código (RCE)**.

Casos reales

En móviles (0-click)

- **FORCEDENTRY / Pegasus (2021):**

OOB write en el decoder de imágenes JBIG2; comprometía un iPhone sin un solo click, vía iMessage.

- **Stagefright (Android, 2015):**

OOB write parseando video de un MMS; ~950 millones de dispositivos afectados.

El patrón de siempre

- Buffer overflow clásico: un input largo pisa un flag de autorización o el return address (lo de la demo).
- Aparece hace décadas y sigue #1: MITRE lo ubica como el CWE más peligroso.
- Vive en librerías de C/C++ que están en todos lados: media, red, parsers.

Out-of-bounds Write

Nombre	Escritura fuera de límites (Out-of-bounds Write) — CWE-787.
Resumen	El software escribe datos más allá del final (o antes del comienzo) del buffer reservado, corrompiendo la memoria adyacente.
Descripción extendida	Ocurre al no validar el tamaño o el índice antes de escribir. La escritura cae en memoria que no corresponde —variables vecinas, dirección de retorno, metadatos— permitiendo corromper datos, cambiar el flujo del programa o ejecutar código.
Forma de introducción	Funciones sin control de límites (strcpy, memcpy, sprintf, gets), índices o punteros calculados sin validar, y errores off-by-one en los tamaños. Típico en C/C++.
Mitigaciones potenciales	Funciones acotadas (strncpy, snprintf) y validar longitud/índice; lenguajes memory-safe (Rust, Go); defensas del compilador (stack canaries, ASLR, FORTIFY_SOURCE) y sanitizers + fuzzing.
Consecuencias comunes	Corrupción de memoria y caídas (DoS); modificación de datos y de decisiones del programa; en el peor caso, ejecución de código arbitrario (RCE).
Plataformas aplicables	Lenguajes sin gestión automática de memoria (C, C++, Assembly). Frecuente en firmware, embebidos, navegadores y librerías de parsing.

Demo en vivo

DEMO

TechStore

CWE-787

Demo en vivo · Out-of-bounds Write (Escritura fuera de límites)

TU CARRITO

Notebook Gamer RTX

16 GB RAM · 1 TB SSD · Envío gratis

\$1.200

Backend VULNERABLE

Backend CORREGIDO

Código de cupón

ej: SALE10

Aplicar

longitud del cupón: 0 bytes · buffer reservado: 16 bytes

Cupón inválido

Cupón válido (25%)

Cupón malicioso

Subtotal

\$1.200

Total a pagar

\$1.200

Pagar pedido

MEMORIA DEL SERVIDOR (20 BYTES)

coupon[16] (buffer)

discount_percent (descuento)

00	00	00	00	00	00	00	00	00	00
.	.	.	.	.	.	.	.	.	.
0	1	2	3	4	5	6	7	8	9
00	00	00	00	00	00	00	00	00	00
.	.	.	.	.	.	.	.	.	.
10	11	12	13	14	15	16	17	18	19

char coupon[16]

int %

discount_percent = 0 % → sin descuento

github.com/juanmato/cwe-787-out-of-bounds-write-demo

UCU

Qué pasa por detrás

Estado normal

- coupon[16] guarda el cupón; al lado, discount_percent = 0.
- Cupón válido "SALE25" → descuento = 25 → total \$900.
- Todo queda dentro de los límites del buffer.

Con el cupón malicioso

- 20 bytes "A" no entran en 16 → se desbordan.
- Los 4 bytes sobrantes pisan discount_percent.
- discount_percent = 1.094.795.585 % → total \$0.
- Nunca hubo un cupón válido: solo escribimos fuera de límite.

El arreglo en el código

Vulnerable

```
char coupon[16];  
strcpy(coupon, entrada);
```

Copia sin límite: si “entrada” supera 16 bytes, escribe fuera del buffer y pisa lo de al lado.

Corregido

```
char coupon[16];  
strncpy(coupon, entrada,  
        sizeof(coupon)-1);  
coupon[15] = 0;    // NUL final
```

La copia está acotada al tamaño del buffer: nunca escribe fuera. El descuento queda intacto.

Mitigaciones y recomendaciones

- **Funciones acotadas:** strncpy, snprintf, memcpy_s. Nunca strcpy / gets / sprintf.
- **Validar SIEMPRE** la longitud y los índices antes de escribir.
- **Lenguajes memory-safe:** Rust o Go; en C++ usar std::string y contenedores.
- **Defensas en profundidad:** stack canaries, ASLR, DEP/NX, FORTIFY_SOURCE.
- **Detección:** análisis estático, AddressSanitizer y fuzzing en el CI.

¿Preguntas?

